

Composición

Un objeto puede estar formado por otros objetos. En lugar de acumular atributos sueltos, la clase se arma con piezas que tienen sentido propio.

Un Alumno...

tiene una

Fecha

tiene un

Domicilio

tiene un

Contacto

La clase Alumno

Todos los datos como atributos sueltos de un tipo básico.

17 atributos en una sola clase

```
class Alumno {  
private:  
    int    legajo;  
    string apellido;  
    string nombre;  
    int    diaNacimiento;  
    int    mesNacimiento;  
    int    anioNacimiento;  
    string calle;  
    string numero;  
    string piso;  
    string departamento;  
    string localidad;  
    string codigoPostal;  
    string telefono;  
    string email;  
    int    diaIngreso;  
    int    mesIngreso;  
    int    anioIngreso;  
public:  
    // ...  
};
```

Un objeto alumno

legajo	48512
apellido	Simón
nombre	Angel
diaNacimiento	2
mesNacimiento	10
anioNacimiento	1986
calle	Los Aromos
numero	742
piso	2
departamento	C
localidad	General Pacheco
codigoPostal	B1617
telefono	11-4740-5566
email	asimon@docentes.frgp.utn.edu.ar
diaIngreso	12
mesIngreso	2
anioIngreso	2025

¿Qué atributos viajan siempre juntos?

Un grupo de atributos que sólo tiene sentido completo es, casi siempre, una clase.

legajo · apellido · nombre



Alumno

Identifican al alumno: se quedan.

diaNacimiento · mesNacimiento ·
anioNacimiento · diaIngreso · mesIngreso
anioIngreso



Fecha

Un día suelto no significa nada. Y aparece dos veces.

calle · numero · piso · departamento
localidad · codigoPostal



Domicilio

Seis atributos que describen un único lugar.

telefono · email



Contacto

Las formas de comunicarse con el alumno.

Antes y después

Los mismos datos, con otra estructura.

Sin refactorizar · 17 atributos

```
class Alumno {  
private:  
    int    legajo;  
    string apellido;  
    string nombre;  
    int    diaNacimiento;  
    int    mesNacimiento;  
    int    anioNacimiento;  
    string calle;  
    string numero;  
    string piso;  
    string departamento;  
    string localidad;  
    string codigoPostal;  
    string telefono;  
    string email;  
    int    diaIngreso;  
    int    mesIngreso;  
    int    anioIngreso;  
public:  
    // ...  
};
```

Refactorizada · 7 atributos

```
class Alumno {  
private:  
    int    legajo;  
    string apellido;  
    string nombre;  
    Fecha  fechaNacimiento;  
    Domicilio domicilio;  
    Contacto contacto;  
    Fecha  fechaIngreso;  
public:  
    // ...  
};
```

```
class Fecha { int dia, mes, anio; };
```

```
class Domicilio {  
    string calle, numero, piso, departamento,  
           localidad, codigoPostal;  
};
```

```
class Contacto { string telefono, email; };
```

No se perdió nada: se organizó mejor

El mismo alumno del ejemplo, ahora con los datos agrupados.

```
class Alumno {  
private:  
    int        legajo;  
    string      apellido;  
    string      nombre;  
    Fecha       fechaNacimiento;  
    Domicilio   domicilio;  
    Contacto    contacto;  
    Fecha       fechaIngreso;  
public:  
    // ...  
};
```

Alumno ya no conoce el día ni el código postal: conoce una Fecha y un Domicilio, y le pide a cada uno lo que necesita.

Cada clase valida sus propios datos: Fecha controla que el mes esté entre 1 y 12.

alumno : Alumno

legajo 48512
apellido Simón
nombre Angel

fechaNacimiento : Fecha

día 2 · mes 10 · año 1986

domicilio : Domicilio

calle Los Aromos · número 742
piso 2 · departamento C
localidad General Pacheco · cp B1617

contacto : Contacto

telefono 11-4740-5566
email asimon@docentes.frgp.utn.edu.ar

fechaIngreso : Fecha

día 12 · mes 2 · año 2025

Plan de refactorización

Seis pasos que sirven para cualquier clase que creció de más.

1 Identificar

Listar todos los atributos y escribir al lado qué representa cada uno.

2 Agrupar

Marcar los que siempre viajan juntos y que por separado no significan nada.

3 Clasificar

Si el grupo se puede nombrar con un sustantivo (Fecha, Domicilio), es una clase.

4 Crear la clase

Atributos privados, constructor que valide y los métodos propios del grupo.

5 Reemplazar

En Alumno, cambiar los atributos sueltos por un objeto del nuevo tipo.

6 Delegar

Adaptar constructor, getters y mostrar() pidiéndole los datos al objeto interno.

¿Y el Domicilio ya está terminado?

El mismo análisis se puede volver a aplicar adentro de la clase que acabamos de crear.

Como quedó

```
class Domicilio {  
    string calle, numero, piso, departamento;  
    string localidad; // el nombre, suelto  
    string codigoPostal; // y su código, suelto  
};
```

Un paso más

```
class Localidad {  
    string nombre, codigoPostal;  
};  
  
class Domicilio {  
    string calle, numero, piso, departamento;  
    Localidad localidad;  
};
```

Un domicilio siempre tiene una localidad, y una localidad siempre tiene un nombre y un código postal. Los dos atributos vuelven a viajar juntos: es otra clase esperando salir.



Actividad: escribir la clase Localidad y componer Domicilio con ella.